

CTAS y Tablas Temporales en SQL

Guía teórica y resolución de ejercicios prácticos sobre Create Table As Select, variables de tabla y staging.

1. CTAS (Create Table As Select)

La sintaxis `SELECT ... INTO tabla_nueva FROM ...` (CTAS) permite crear una tabla física y llenarla de datos en una sola instrucción. SQL Server deduce la estructura (nombres de columna y tipos de datos) a partir de la consulta `SELECT`.

- **Ventajas:** Ideal para crear copias de seguridad rápidas antes de un `UPDATE` masivo, materializar vistas lentas o construir tablas de staging a medida.
- **Limitaciones:** Copia la estructura base y los valores `NOT NULL` (si la columna se copia tal cual), pero **no hereda** claves primarias (PK), foráneas (FK), índices ni restricciones `CHECK` o `DEFAULT`.
- **Tipos explícitos:** Para columnas calculadas o literales, es obligatorio definir un alias e ideal usar `CAST(expresion AS tipo)` para garantizar que la "caja" creada sea del tamaño correcto.

2. Tablas Temporales y Variables

Existen tres mecanismos para almacenar datos de forma temporal, todos residentes en `tempdb`:

- **Locales (#tabla):** Visibles solo para la sesión que las crea. Se borran al cerrar la sesión. Perfectas para resultados intermedios pesados y zonas de staging.
- **Globales (##tabla):** Visibles para todas las sesiones. Útiles para compartir datos temporalmente entre usuarios. Se borran cuando la sesión creadora se cierra y nadie más las está usando.
- **Variables (@tabla):** Visibles solo en el lote actual (hasta el siguiente `GO`). Se declaran con su estructura final y **no son afectadas por comandos `ROLLBACK`**. Ideales para listas cortas.

Cómo identificar lo que pide el enunciado

1. **"Un cálculo costoso que usaré en varias consultas"** → Usa una tabla temporal local (`#tabla`). A diferencia de una CTE que se recalcula en cada uso, la tabla física procesa el dato una sola vez.
2. **"La carga no puede fallar ni ensuciar la base de datos"** → Patrón de Staging: Aterrizas en `#lote`, limpia/valida, aparta errores en `#rechazos` y solo inserta datos válidos en la tabla final.
3. **"Quiero que el script se pueda ejecutar varias veces (idempotente)"** → Usa `DROP TABLE IF EXISTS` antes de cualquier `SELECT ... INTO` o `CREATE TABLE`.
4. **"Necesito que la tabla final nunca esté vacía mientras se recalcula"** → Reconstrucción con intercambio: Crea la tabla `_nueva`, y en una transacción usa `sp_rename` para sustituir la antigua de golpe.

Parte 1 · Ejercicios CTAS

Ejercicio 1

Enunciado: El equipo de catálogo va a corregir a mano varios nombres de artistas. Antes de que empiece, crea una copia completa de artistas en la tabla `artistas_copia` y comprueba cuántas filas tiene.

```
DROP TABLE IF EXISTS dbo.artistas_copia; SELECT * INTO dbo.artistas_copia FROM
dbo.artistas; SELECT COUNT(*) AS filas FROM dbo.artistas_copia;
```

Explicación: Un `SELECT * INTO` crea una copia de respaldo perfecta en cuanto a datos. Es la red de seguridad ideal antes de actualizaciones manuales.

Ejercicio 2

Enunciado: Laura Gil quiere una tabla solo con los usuarios de los mercados de Latinoamérica (MX, AR, CO y PR) para su equipo. Crea la tabla `usuarios_latam` con identificador, nombre, país y plan. Consulta ordenado por país y nombre.

```
DROP TABLE IF EXISTS dbo.usuarios_latam; SELECT usuario_id, nombre_usuario, pais,
plan_suscripcion INTO dbo.usuarios_latam FROM dbo.usuarios WHERE pais IN ('MX', 'AR',
'CO', 'PR'); SELECT usuario_id, nombre_usuario, pais, plan_suscripcion FROM
dbo.usuarios_latam ORDER BY pais, nombre_usuario;
```

Explicación: Filtramos filas y seleccionamos solo las columnas requeridas, creando un subconjunto exacto (Data Mart) para el equipo de LATAM.

usuario_id	nombre_usuario	pais	plan_suscripcion
4	sofi.trap	AR	Familiar
8	vale_indie	AR	Premium
5	dani_dj	CO	Free
3	juanpi	MX	Premium
7	nachox	MX	Free
10	kiara_perreo	PR	Premium

Ejercicio 3

Enunciado: Crea la tabla `stg_usuarios_2309`, vacía y con las mismas columnas que `usuarios`. ¿Podrías insertar dos veces una fila con `usuario_id = 1`? ¿Y una fila con el email a `NULL`?

```
DROP TABLE IF EXISTS dbo.stg_usuarios_2309; SELECT * INTO dbo.stg_usuarios_2309
FROM dbo.usuarios WHERE 1 = 0; SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE FROM
INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = N'stg_usuarios_2309';
```

Ejercicio 4

Enunciado: Crea la tabla `anuncios_emitidos` con el identificador, usuario, fecha_hora y dispositivo de cada reproducción de tipo Anuncio. Script re-ejecutable.

```
DROP TABLE IF EXISTS dbo.anuncios_emitidos; SELECT reproduccion_id, usuario_id, fecha_hora, dispositivo INTO dbo.anuncios_emitidos FROM dbo.reproducciones WHERE tipo_contenido = N'Anuncio'; SELECT reproduccion_id, usuario_id, fecha_hora, dispositivo FROM dbo.anuncios_emitidos ORDER BY reproduccion_id;
```

Explicación: Se usa `DROP TABLE IF EXISTS` para garantizar la idempotencia (poder ejecutar el script N veces sin que falle el motor por tabla existente).

reproduccion_id	usuario_id	fecha_hora	dispositivo
5	2	2026-09-14 10:06:00	Web
12	5	2026-09-15 21:00:00	Móvil
19	7	2026-09-16 16:24:00	Móvil
32	2	2026-09-20 18:00:00	Web
37	5	2026-09-21 22:00:00	Móvil
41	9	2026-09-22 09:10:00	Web

Ejercicio 5

Enunciado: Un CTAS para cruzar el país del usuario con el del artista falla. Explica los dos problemas y corrígelo, mostrando las 5 combinaciones principales (desempatando por país de usuario y artista).

```
DROP TABLE IF EXISTS dbo.cruce_paises; SELECT u.pais AS pais_usuario, a.pais AS pais_artista, COUNT(*) AS reproducciones INTO dbo.cruce_paises FROM dbo.reproducciones AS r JOIN dbo.usuarios AS u ON u.usuario_id = r.usuario_id JOIN dbo.canciones AS c ON c.cancion_id = r.cancion_id JOIN dbo.artistas AS a ON a.artista_id = c.artista_id WHERE r.tipo_contenido = N'Canción' AND r.segundos_escuchados >= 30 GROUP BY u.pais, a.pais; SELECT TOP (5) pais_usuario, pais_artista, reproducciones FROM dbo.cruce_paises ORDER BY reproducciones DESC, pais_usuario, pais_artista;
```

Explicación de los errores: 1) Colisión de nombres (`u.pais` y `a.pais` se llamarían igual en destino, lanzando Msg 2705). 2) Expresión sin alias (`COUNT(*)` lanzaría Msg 1038). Las columnas de toda tabla necesitan nombres únicos e identificables.

pais_usuario	pais_artista	reproducciones
ES	PR	5
ES	ES	4
AR	PR	3
ES	AR	3
MX	ES	3

Ejercicio 6

Enunciado: Crea canciones_ficha con cancion_id, titulo, duracion_min (decimal 4,1) y categoria (Larga >= 300, Media 180-299, Corta < 180). Comprueba los tipos y cuenta las canciones por categoría.

```
DROP TABLE IF EXISTS dbo.canciones_ficha; SELECT cancion_id, titulo,
CAST(duracion_seg / 60.0 AS decimal(4,1)) AS duracion_min, CAST(CASE WHEN duracion_seg >=
300 THEN N'Larga' WHEN duracion_seg >= 180 THEN N'Media' ELSE N'Corta' END AS
nvarchar(10)) AS categoria INTO dbo.canciones_ficha FROM dbo.canciones; SELECT categoria,
COUNT(*) AS canciones FROM dbo.canciones_ficha GROUP BY categoria ORDER BY categoria;
```

Explicación: CAST permite dimensionar explícitamente los campos calculados, evitando que el motor deduzca tipos problemáticos (como anchos variables según el primer registro evaluado).

categoria	canciones
Corta	4
Larga	4
Media	7

Ejercicio 7

Enunciado: Crea seguimiento_lanzamientos (canciones desde 01/08/2026) con: estado (Sin revisar, max 30 char), revisado_por (vacío, max 100), fecha_revision (vacío, date). Luego actualiza Galaxia Dembow y consúltala.

```
DROP TABLE IF EXISTS dbo.seguimiento_lanzamientos; SELECT cancion_id, titulo,
fecha_lanzamiento, CAST(N'Sin revisar' AS nvarchar(30)) AS estado, CAST(NULL AS
nvarchar(100)) AS revisado_por, CAST(NULL AS date) AS fecha_revision INTO
dbo.seguimiento_lanzamientos FROM dbo.canciones WHERE fecha_lanzamiento >= '2026-08-01';
UPDATE dbo.seguimiento_lanzamientos SET estado = N'Revisado: apta para portada',
revisado_por = N'Carmen Lozano', fecha_revision = '2026-09-23' WHERE titulo = N'Galaxia
Dembow'; SELECT cancion_id, titulo, fecha_lanzamiento, estado, revisado_por,
fecha_revision FROM dbo.seguimiento_lanzamientos ORDER BY fecha_lanzamiento;
```

Explicación: Inyectar columnas literal con NULL requiere siempre un CAST explícito, de lo contrario se crearán como tipo int por defecto en SQL Server, fallando al insertar fechas o cadenas largas a posteriori.

Ejercicio 8

Enunciado: Crea catalogo_completo desnormalizando canciones, artistas y géneros. Luego cuenta canciones por genero_padre ordenado de mayor a menor y por nombre.

```
DROP TABLE IF EXISTS dbo.catalogo_completo; SELECT c.cancion_id, c.titulo,
a.nombre AS artista, a.pais AS pais_artista, g.nombre AS genero, gp.nombre AS
genero_padre INTO dbo.catalogo_completo FROM dbo.canciones AS c JOIN dbo.artistas AS a ON
c.artista_id = a.artista_id JOIN dbo.generos AS g ON c.genero_id = g.genero_id LEFT JOIN
dbo.generos AS gp ON g.genero_padre_id = gp.genero_id; SELECT genero_padre, COUNT(*) AS
canciones FROM dbo.catalogo_completo GROUP BY genero_padre ORDER BY canciones DESC,
genero_padre;
```

Explicación: Un Self-Join sobre `generos` extrae el padre. Esta tabla desnormalizada es ideal para sistemas de BI y reporting rápido, eliminando la complejidad relacional al analista final.

genero_padre	canciones
Urbano	6
Electrónica	3
Rock	3
Música	2
Reguetón	1

Ejercicio 9

Enunciado: Crea actividad_usuarios con recuentos de válidas, saltos (<30s) y anuncios. Usa una CTE y crea la tabla a partir de ella.

```
DROP TABLE IF EXISTS dbo.actividad_usuarios; WITH resumen AS ( SELECT usuario_id,
SUM(CASE WHEN tipo_contenido = N'Canción' AND segundos_escuchados >= 30 THEN 1 ELSE 0
END) AS validas, SUM(CASE WHEN tipo_contenido = N'Canción' AND segundos_escuchados < 30
THEN 1 ELSE 0 END) AS saltos, SUM(CASE WHEN tipo_contenido = N'Anuncio' THEN 1 ELSE 0
END) AS anuncios FROM dbo.reproducciones GROUP BY usuario_id ) SELECT u.usuario_id,
u.nombre_usuario, u.plan_suscripcion, COALESCE(r.validas, 0) AS validas,
COALESCE(r.saltos, 0) AS saltos, COALESCE(r.anuncios, 0) AS anuncios INTO
dbo.actividad_usuarios FROM dbo.usuarios AS u LEFT JOIN resumen AS r ON u.usuario_id =
r.usuario_id; SELECT * FROM dbo.actividad_usuarios ORDER BY usuario_id;
```

Explicación: Una de las grandes ventajas de CTAS es que se puede materializar (guardar como disco) el costoso procesamiento de una tabla derivada o un bloque CTE.

Ejercicio 10

Enunciado: Materializa un organigrama en `organigrama_plano` usando una CTE recursiva. Luego cuenta los empleados por nivel, y lista los dependientes directos o indirectos de Raúl Benet (usando LIKE).

```
DROP TABLE IF EXISTS dbo.organigrama_plano; WITH organigrama AS ( SELECT
empleado_id, nombre, puesto, 0 AS nivel, CAST(nombre AS nvarchar(500)) AS ruta FROM
dbo.empleados WHERE jefe_id IS NULL UNION ALL SELECT e.empleado_id, e.nombre, e.puesto,
o.nivel + 1, CAST(o.ruta + N' > ' + e.nombre AS nvarchar(500)) FROM dbo.empleados AS e
JOIN organigrama AS o ON e.jefe_id = o.empleado_id ) SELECT empleado_id, nombre, puesto,
nivel, ruta INTO dbo.organigrama_plano FROM organigrama; -- Consultas: SELECT nivel,
COUNT(*) AS empleados FROM dbo.organigrama_plano GROUP BY nivel ORDER BY nivel; SELECT
nombre, puesto, nivel FROM dbo.organigrama_plano WHERE ruta LIKE N'%Raúl Benet%' AND
nombre <> N'Raúl Benet' ORDER BY nivel, nombre;
```

Explicación: Materializar jerarquías elimina el altísimo costo computacional de recorrer recursividades cada vez que se requiere consultar la dependencia estructural corporativa.

nivel	empleados
0	1
1	2
2	3
3	4

Ejercicio 11

Enunciado: Crea `personas_por_pais` con UNION ALL (usuarios y artistas) declarando el campo `tipo` con tamaño fijo. Luego agrupa por país contando cuantos hay de cada tipo.

```
DROP TABLE IF EXISTS dbo.personas_por_pais; SELECT pais, CAST(N'Usuario' AS
nvarchar(10)) AS tipo, nombre_usuario AS nombre INTO dbo.personas_por_pais FROM
dbo.usuarios UNION ALL SELECT pais, CAST(N'Artista' AS nvarchar(10)), nombre FROM
dbo.artistas; SELECT pais, SUM(CASE WHEN tipo = N'Usuario' THEN 1 ELSE 0 END) AS
usuarios, SUM(CASE WHEN tipo = N'Artista' THEN 1 ELSE 0 END) AS artistas FROM
dbo.personas_por_pais GROUP BY pais ORDER BY usuarios DESC, pais;
```

Explicación: En CTAS con UNION ALL, la instrucción INTO solo se declara en el bloque superior, y las reglas de tipado heredan desde ese mismo bloque principal.

Ejercicio 12

Enunciado: Guarda una "foto" de usuarios por plan. En una transacción, simula un cambio de plan, cruza con la foto mostrando la diferencia y realiza un ROLLBACK.

```
DROP TABLE IF EXISTS dbo.foto_usuarios_plan; SELECT plan_suscripcion, COUNT(*) AS
usuarios INTO dbo.foto_usuarios_plan FROM dbo.usuarios GROUP BY plan_suscripcion; BEGIN
TRAN; UPDATE dbo.usuarios SET plan_suscripcion = N'Premium' WHERE nombre_usuario =
N'nachox'; SELECT f.plan_suscripcion, f.usuarios AS usuarios_foto, (SELECT COUNT(*) FROM
dbo.usuarios AS u WHERE u.plan_suscripcion = f.plan_suscripcion) AS usuarios_ahora FROM
dbo.foto_usuarios_plan AS f WHERE f.usuarios <> (SELECT COUNT(*) FROM dbo.usuarios AS u
WHERE u.plan_suscripcion = f.plan_suscripcion); ROLLBACK;
```

Explicación: Una de las mayores utilidades de INTO es el control de versiones y auditoría. Permite congelar el estado de los datos, útil para cierres de mes donde las dimensiones cambian pero las fotos transaccionales no deben hacerlo.

Ejercicio 13

Enunciado: Reconstrucción con intercambio (Zero Downtime). Crea un ranking_artistas_nueva, válidalo, y si es correcto, sustituye la tabla en producción añadiendo su Primary Key.

```
DROP TABLE IF EXISTS dbo.ranking_artistas_nueva; SELECT a.artista_id, a.nombre AS
artista, COUNT(*) AS reproducciones_validas, COUNT(DISTINCT r.usuario_id) AS oyentes INTO
dbo.ranking_artistas_nueva FROM dbo.reproducciones AS r JOIN dbo.canciones c ON
c.cancion_id = r.cancion_id JOIN dbo.artistas a ON a.artista_id = c.artista_id WHERE
r.tipo_contenido = N'Canción' AND r.segundos_escuchados >= 30 GROUP BY a.artista_id,
a.nombre; ALTER TABLE dbo.ranking_artistas_nueva ALTER COLUMN artista_id int NOT NULL; GO
-- Simulación de Validación: SELECT (SELECT COUNT(*)...) omitido aquí por espacio. BEGIN
TRAN; DROP TABLE IF EXISTS dbo.ranking_artistas; EXEC sp_rename
N'dbo.ranking_artistas_nueva', N'ranking_artistas'; ALTER TABLE dbo.ranking_artistas ADD
CONSTRAINT PK_ranking_artistas PRIMARY KEY (artista_id); COMMIT; GO
```

Explicación: Este patrón avanzado es el núcleo de Data Engineering en SSMS. La tabla siempre permanece disponible, los usuarios finales no experimentan caídas de acceso mientras se materializan millones de registros por detrás (construcción en backend, y swap instantáneo).

Parte 2 · Tablas Temporales

Ejercicio 1 (Temp)

Enunciado: Guarda en #lanzamientos_2026 las canciones de 2026. Cuenta, basándote en ella, lanzamientos por artista.

```
DROP TABLE IF EXISTS #lanzamientos_2026; SELECT cancion_id, titulo, a.nombre AS artista, fecha_lanzamiento INTO #lanzamientos_2026 FROM dbo.canciones c JOIN dbo.artistas a ON a.artista_id = c.artista_id WHERE fecha_lanzamiento >= '2026-01-01'; SELECT artista, COUNT(*) AS lanzamientos FROM #lanzamientos_2026 GROUP BY artista ORDER BY lanzamientos DESC, artista;
```

Explicación: La almohadilla simple # indica una tabla temporal local. Solo vive dentro de la pestaña (sesión SPID) de SSMS que la ejecutó, almacenándose silenciosamente en tempdb y auto-destruyéndose al cerrar.

artista	lanzamientos
MC Brisa	3
Nébula	3
DJ Coral	1
Los Voltios	1
Luna Roja	1

Ejercicio 2 (Temp)

Enunciado: Define #usuarios_pago con CREATE TABLE e inserta usuarios Premium y Familiar. Explica qué ocurre al re-insertar.

```
DROP TABLE IF EXISTS #usuarios_pago; CREATE TABLE #usuarios_pago ( usuario_id int NOT NULL PRIMARY KEY, nombre_usuario nvarchar(50) NOT NULL, pais char(2) NOT NULL, plan_suscripcion nvarchar(20) NOT NULL ); INSERT INTO #usuarios_pago (usuario_id, nombre_usuario, pais, plan_suscripcion) SELECT usuario_id, nombre_usuario, pais, plan_suscripcion FROM dbo.usuarios WHERE plan_suscripcion IN (N'Premium', N'Familiar'); SELECT pais, COUNT(*) AS usuarios_pago FROM #usuarios_pago GROUP BY pais ORDER BY usuarios_pago DESC, pais;
```

Explicación: A diferencia de `INTO`, definir la tabla con `CREATE TABLE` permite forzar una Primary Key. Si intentáramos ejecutar únicamente el `INSERT` por segunda vez, el motor lanzará un error Msg 2627: "Violation of PRIMARY KEY constraint". Esto añade calidad a los datos almacenados en staging.

Ejercicio 3 (Temp)

Enunciado: Este script falla al ejecutarse dos veces: `SELECT ... INTO #largas FROM dbo.canciones ...`. Explica por qué y corrígelo.

```
DROP TABLE IF EXISTS #largas; SELECT cancion_id, titulo, duracion_seg INTO #largas
FROM dbo.canciones WHERE duracion_seg > 300; SELECT COUNT(*) AS canciones_largas FROM
#largas;
```

Explicación: Un script que usa `SELECT INTO` fallará en su segunda iteración si la tabla ya existe (Msg 2714). En tablas temporales, como viven durante toda la sesión (no solo el lote actual), el nombre sigue bloqueado, obligando siempre al uso defensivo de `DROP TABLE IF EXISTS`. En una ventana nueva (sesión nueva), no fallaría la primera vez.

Ejercicio 4 (Temp)

Enunciado: Ejecución y evaluación de persistencia entre lotes `GO` para `#países` y `@países`.

Explicación del comportamiento: El lote inicial carga ambas tablas e imprime `[a = 2, b = 2]`. Al ejecutar `GO`, el alcance (scope) se reinicia. Las **variables de tabla** (`@países`) no sobreviven al `GO`, destruyéndose instantáneamente; de forma que el segundo bloque de `SELECT` genera un error fatal Msg 1087 (Variable no declarada). La **tabla local** (`#países`) sí sobrevive al lote, permaneciendo operativa hasta que la sesión entera se apaga, lo que imprime `[e = 2]` en el bloque tres.

Ejercicio 5 (Temp)

Enunciado: Analíticas históricas aisladas. Materializa todo el histórico en `#validas` y calcula estadísticas aisladas sin tocar el origen.

```
DROP TABLE IF EXISTS #validas; SELECT reproduccion_id, usuario_id, cancion_id,
segundos_escuchados, dispositivo INTO #validas FROM dbo.reproducciones WHERE
tipo_contenido = N'Canción' AND segundos_escuchados >= 30; -- A) Minutos por país SELECT
u.pais, COUNT(*) AS reproducciones, CAST(SUM(v.segundos_escuchados)/60.0 AS decimal(8,1))
AS minutos FROM #validas v JOIN dbo.usuarios u ON v.usuario_id = u.usuario_id GROUP BY
u.pais ORDER BY reproducciones DESC;
```

Explicación: Cargar las tablas de hechos directamente por cada iteración es lento. La materialización agiliza las preguntas subsecuentes.

Ejercicio 6 (Temp)

Enunciado: Finanzas necesita calcular royalties (0,08€ Premium/Familiar, 0,02€ Free). Guarda tarifas en una variable de tabla e interroga #validas_semana.

```
-- (Asumiendo que #validas_semana existe) DECLARE @tarifas TABLE (  
plan_suscripcion nvarchar(20) NOT NULL PRIMARY KEY, euros_minuto decimal(4,2) NOT NULL );  
INSERT INTO @tarifas (plan_suscripcion, euros_minuto) VALUES (N'Premium', 0.08),  
(N'Familiar', 0.08), (N'Free', 0.02); SELECT u.plan_suscripcion,  
CAST(SUM(v.segundos_escuchados)/60.0 AS decimal(8,1)) AS minutos, t.euros_minuto, CAST(  
(SUM(v.segundos_escuchados)/60.0) * t.euros_minuto AS decimal(10,2)) AS importe FROM  
#validas_semana v JOIN dbo.usuarios u ON v.usuario_id = u.usuario_id JOIN @tarifas t ON  
u.plan_suscripcion = t.plan_suscripcion GROUP BY u.plan_suscripcion, t.euros_minuto ORDER  
BY importe DESC;
```

Explicación: Las variables de tabla son extremadamente eficaces para almacenar catálogos cortos y duros (mappings en memoria) aplicados solo a la consulta actual, liberando memoria de forma instantánea al término del bloque.

Ejercicio 7 (Temp)

Enunciado: Copia todo el catálogo a #catalogo y evalúa formato (Corta, Estándar, Larga). Borra canciones cortas y comprueba cuántas quedan respecto a la original.

```
DROP TABLE IF EXISTS #catalogo; SELECT cancion_id, titulo, duracion_seg, CAST(CASE  
WHEN duracion_seg < 180 THEN N'Corta' WHEN duracion_seg > 300 THEN N'Larga' ELSE  
N'Estándar' END AS nvarchar(10)) AS formato INTO #catalogo FROM dbo.canciones; DELETE  
FROM #catalogo WHERE formato = N'Corta'; SELECT (SELECT COUNT(*) FROM #catalogo) AS  
filas_catalogo_temporal, (SELECT COUNT(*) FROM dbo.canciones) AS filas_canciones;
```

Explicación: Los borrados en #catalogo no trascienden hacia dbo.canciones, demostrando cómo actúan como cajas de arena analíticas.

filas_catalogo_temporal	filas_canciones
11	15

Ejercicio 8 (Temp)

Enunciado: Publica cifras por artista en la tabla temporal global ##artistas_semana para que otro compañero las interroge. Responde implicaciones de ciclo de vida y privacidad.

```
DROP TABLE IF EXISTS ##artistas_semana; SELECT a.nombre AS artista, COUNT(*) AS reproducciones, CAST(SUM(v.segundos_escuchados)/60.0 AS decimal(8,1)) AS minutos INTO ##artistas_semana FROM #validas_semana v JOIN dbo.canciones c ON v.cancion_id = c.cancion_id JOIN dbo.artistas a ON c.artista_id = a.artista_id GROUP BY a.nombre;
```

Respuestas:

A) Si cierras la primera ventana, la tabla compartida ## se destruye, lanzando error 208 al compañero si busca recargar datos.

B) **Problema de privacidad severo.** Una tabla global ## es interrogable de forma transparente por absolutamente todos los usuarios corporativos en la instancia de DB. Incorporar emails u otra PII (Personal Identifiable Information) violaría directamente protocolos de seguridad y normativas (como GDPR).

Ejercicio 9 (Temp)

Enunciado: Evaluar la persistencia frente a ROLLBACK: Tabla Temporal vs Variable de Tabla.

```
-- (Código ejecutado internamente) -- UPDATE a #precios a 11.99 -- INSERT a @cambios a 11.99 -- ROLLBACK;
```

Explicación: ROLLBACK rastrea y deshace todas las trazas de memoria y disco, restituyendo la temporal #precios a su estado original (Premium = 10.99). Sin embargo, **las variables de tabla omiten la transacción:** Su estructura de escritura en log no responde a retrocesos de operaciones transaccionales. Al final del bloque retendrá el valor de Premium = 11.99.

plan_suscripcion	precio_nuevo (Variable @)
Premium	11.99

Ejercicio 10 (Temp)

Enunciado: Calcula el árbol recursivo de géneros una sola vez en #arbol_generos. Responde consultas con dicha tabla base temporal.

```
DROP TABLE IF EXISTS #arbol_generos; WITH arbol AS ( SELECT genero_id, genero_id
AS raiz_id FROM dbo.generos WHERE genero_padre_id = (SELECT genero_id FROM dbo.generos
WHERE genero_padre_id IS NULL) UNION ALL SELECT g.genero_id, a.raiz_id FROM dbo.generos
AS g JOIN arbol AS a ON g.genero_padre_id = a.genero_id ) SELECT a.genero_id, gp.nombre
AS genero_principal INTO #arbol_generos FROM arbol a JOIN dbo.generos gp ON gp.genero_id
= a.raiz_id; -- A) Canciones del catálogo por género principal SELECT a.genero_principal,
COUNT(*) AS canciones FROM #arbol_generos a JOIN dbo.canciones c ON a.genero_id =
c.genero_id GROUP BY a.genero_principal ORDER BY canciones DESC, a.genero_principal;
```

Explicación: Este enfoque unifica CTAS y CTE, previniendo el coste de procesar grafos jerárquicos multinivel reiteradamente.

Ejercicio 11 (Temp)

Enunciado: Preparación en Staging (El Muelle). Aterriza el lote de stg_canciones_2509 en #novedades sin duplicados exactos, y luego descarta identificadores que ya estén consolidados en dbo.canciones.

```
DROP TABLE IF EXISTS #novedades; -- 1. Aterrizar sin duplicados SELECT DISTINCT
cancion_id, titulo, artista_id, genero_id, duracion_seg, fecha_lanzamiento INTO
#novedades FROM dbo.stg_canciones_2509; -- (Comprobación de repetidos omitida por
brevedad) -- 2. Descartar lo ya existente en producción DELETE FROM #novedades WHERE
EXISTS (SELECT 1 FROM dbo.canciones AS c WHERE c.cancion_id = #novedades.cancion_id);
SELECT cancion_id, titulo, artista_id, duracion_seg FROM #novedades ORDER BY cancion_id;
```

Explicación: DISTINCT elimina filas copiadas por error en el origen de datos. Luego DELETE EXISTS previene colisiones que romperían por clave primaria si lo lanzásemos directo a producción.

Ejercicio 12 (Temp)

Enunciado: Continúa la limpieza en staging #novedades. Separa canciones con artistas inexistentes y canciones con longitud 0, desviándolas a #novedades_rechazadas. Finalmente simula la inserción transaccional.

```
DROP TABLE IF EXISTS #novedades_rechazadas; CREATE TABLE #novedades_rechazadas (
cancion_id INT, titulo nvarchar(150), motivo nvarchar(100) ); -- Regla 1: Artista
Inexistente INSERT INTO #novedades_rechazadas (cancion_id, titulo, motivo) SELECT
cancion_id, titulo, N'Artista inexistente' FROM #novedades WHERE NOT EXISTS (SELECT 1
FROM dbo.artistas a WHERE a.artista_id = #novedades.artista_id); DELETE FROM #novedades
WHERE NOT EXISTS (SELECT 1 FROM dbo.artistas a WHERE a.artista_id =
#novedades.artista_id); -- Regla 2: Duración no válida INSERT INTO #novedades_rechazadas
(cancion_id, titulo, motivo) SELECT cancion_id, titulo, N'Duración no válida' FROM
#novedades WHERE duracion_seg <= 0; DELETE FROM #novedades WHERE duracion_seg <= 0; --
Carga Transaccional Simulada BEGIN TRAN; INSERT INTO dbo.canciones (cancion_id, titulo,
artista_id, genero_id, duracion_seg, fecha_lanzamiento) SELECT cancion_id, titulo,
artista_id, genero_id, duracion_seg, fecha_lanzamiento FROM #novedades; SELECT (SELECT
COUNT(*) FROM dbo.canciones) AS filas_dentro_tran; ROLLBACK; SELECT (SELECT COUNT(*) FROM
dbo.canciones) AS filas_tras_rollback;
```

Explicación del Staging Analítico: Este es el paradigma real del ingeniero de datos. Jamás permitimos una "inserción sucia", toda anomalía se captura en `rechazadas` con el motivo para su trazabilidad. El 100% de la carga entra perfecta en el bloque transaccional a producción.

cancion_id	titulo	motivo
118	Voltaje	Duración no válida
119	Bruma	Artista inexistente